DATABASE MANAGEMENT SYSTEM

DBMS – Complete Notes: All Major Topics

Simple English | Definitions | Key Points | SQL | Examples | Exam Revision

For BCA / MCA / Computer Science Students

This is a broad study guide. Chapter names and syllabus coverage can vary by university; compare it with your official syllabus.

Chapter 1: Introduction to DBMS

Data: raw facts. **Information:** processed data with meaning. **Database:** an organized collection of related data. **DBMS:** software used to define, create, store, retrieve, update and manage a database.

Examples of DBMS

- Relational DBMS: MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server, SQLite.
- Other database systems include MongoDB (document-oriented) and Redis (key-value/in-memory use cases).

Why use a DBMS?

- Reduce unnecessary data redundancy and inconsistency.
- Share data among multiple authorized users and applications.
- Enforce integrity constraints and access permissions.
- Support backup, recovery and concurrent access.
- Provide data independence and centralized management.

File system vs DBMS

File system	DBMS
Applications manage files and data formats separately.	Provides structured data management and query facilities.
Redundancy and inconsistency can be harder to control.	Can reduce redundancy and enforce constraints.
Concurrency, security and recovery often require application-specific work.	Provides built-in mechanisms for concurrency, access control and recovery.

DBMS users and roles

- DBA (Database Administrator): manages users, security, backup, recovery, tuning and availability.
- Database designer: designs schemas, relationships and constraints.
- Application developer: builds software that accesses data.
- End users: query or update data through applications or tools.

Chapter 2: DBMS Architecture and Data Independence

Three-schema architecture

- External level: user-specific views of the data.
- Conceptual level: overall logical structure of the complete database.
- Internal level: how data is physically stored, indexed and organized.

Data independence

- **Physical data independence:** changes to internal storage do not require changes to the conceptual schema or applications, within the architecture's limits.
- **Logical data independence:** changes to the conceptual schema can be isolated from external views and applications where possible.

DBMS components

- Query processor: parses, validates, optimizes and executes queries.
- Storage manager: manages files, indexes, buffers and disk space.
- Transaction manager: coordinates transactions and concurrency.
- Recovery manager: restores consistency after failures.
- Catalog/data dictionary: stores metadata such as tables, columns, constraints and privileges.

Deployment architectures

- 1-tier: user/application works directly with the database environment.
- 2-tier: client application communicates with a database server.
- 3-tier: presentation layer, application/business layer and database layer are separated.

Chapter 3: Data Models and Database Languages

Data models

- Hierarchical model: organizes records in a tree-like parent-child structure.
- Network model: supports more complex links among records.
- Relational model: represents data as relations (tables).
- Entity-Relationship model: models entities, attributes and relationships during design.
- Object-oriented model: represents data using objects and related concepts.
- Document, key-value, graph and column-family models are common NoSQL approaches.

Database languages

- DDL (Data Definition Language): defines structures. Examples: CREATE, ALTER, DROP, TRUNCATE.
- DML (Data Manipulation Language): changes data. Examples: INSERT, UPDATE, DELETE. SELECT is commonly categorized as DQL, though some texts include it in DML.
- DQL (Data Query Language): retrieves data. Example: SELECT.
- DCL (Data Control Language): manages privileges. Examples: GRANT, REVOKE.
- TCL (Transaction Control Language): manages transactions. Examples: COMMIT, ROLLBACK, SAVEPOINT.

Schema and instance

- Schema: the database's defined structure; changes relatively infrequently.
- Instance/state: the actual data stored at a particular time.

Chapter 4: ER Model (Entity–Relationship Model)

The ER model is a conceptual design method used to represent data and relationships before implementing tables.

Key terms

- Entity: distinguishable real-world object, such as Student or Course.
- Entity set: collection of similar entities.
- Attribute: property of an entity, such as StudentName.
- Relationship: association between entities, such as Student ENROLLS in Course.
- Key attribute: attribute that uniquely identifies an entity.

Attribute types

- Simple/atomic: cannot be meaningfully subdivided (e.g., Age).
- Composite: has components (e.g., Address may contain City and PIN).
- Single-valued: one value per entity (e.g., DateOfBirth).
- Multivalued: multiple values (e.g., phone numbers).
- Derived: computed from another attribute (e.g., Age from DateOfBirth).
- Optional/null: may be unknown, missing or not applicable.

Relationship cardinality

- One-to-one (1:1): one entity is related to at most one entity on the other side.
- One-to-many (1:N): one entity can relate to many entities.
- Many-to-many (M:N): many entities can relate to many others; commonly implemented with a junction table.

Participation and weak entities

- Total participation: every entity in the set must participate in the relationship.
- Partial participation: participation is optional for some entities.
- Weak entity: cannot be uniquely identified by its own attributes alone; uses an owner entity's key plus a partial key.

ER-to-relational mapping basics

- Strong entity → table with its key.
- Composite attribute → store its component attributes.
- Multivalued attribute → separate table containing the owner key and value.
- 1:N relationship → place a foreign key on the N-side, when appropriate.
- M:N relationship → create a junction table with foreign keys to both entities.

Chapter 5: Relational Model, Keys and Constraints

Relational terminology

- Relation = table; tuple = row; attribute = column; domain = permitted set of values.
- Degree = number of columns; cardinality = number of rows.
- NULL represents a missing, unknown or inapplicable value; it is not the same as zero or an empty string.

Types of keys

- Super key: any attribute set that uniquely identifies a row.
- Candidate key: minimal super key.
- Primary key: chosen candidate key; must be unique and not NULL.
- Alternate key: candidate key not selected as primary.
- Foreign key: attribute(s) referencing a candidate/primary key in another or the same table.
- Composite key: key made from multiple attributes.
- Surrogate key: artificial identifier, such as an auto-generated ID.

Integrity constraints

- Domain constraint: values must belong to the attribute's valid domain.
- Entity integrity: primary key values cannot be NULL.
- Referential integrity: foreign-key values must match a referenced key, or be NULL where allowed.
- UNIQUE: prevents duplicate values for the constrained column or column set.
- NOT NULL: disallows NULL.
- CHECK: enforces a Boolean condition on values.
- DEFAULT: supplies a value when one is not provided.

Chapter 6: Relational Algebra and Relational Calculus

Relational algebra is a procedural query language: it describes operations used to derive a result relation.

Core relational algebra operations

- Selection (σ): chooses rows satisfying a condition.
- Projection (π): chooses columns; duplicate elimination is part of classical relational algebra.
- Union (∪): combines compatible relations.
- Set difference (–): rows in one relation but not the other.
- Cartesian product (×): pairs each row of one relation with each row of another.
- Rename (ρ): renames a relation or its attributes.
- Join (■): combines related rows using a condition; includes theta, equi and natural joins.
- Intersection (∩): rows common to compatible relations.
- Division (÷): useful for queries expressing “for all” conditions.

Relational calculus

- Tuple relational calculus (TRC) describes queries using tuple variables and logical predicates.
- Domain relational calculus (DRC) uses domain/attribute variables.
- Relational algebra is procedural; relational calculus is declarative.

Chapter 7: SQL – Basics and Table Operations

SQL (Structured Query Language) is widely used to define, query and manipulate relational databases. Exact syntax varies slightly by DBMS.

Example table

Assume a table Student(StudentID, Name, Department, Marks, City).

Create a table

```
CREATE TABLE Student (
StudentID INT PRIMARY KEY,
Name VARCHAR(50) NOT NULL,
Department VARCHAR(30),
Marks DECIMAL(5,2) CHECK (Marks BETWEEN 0 AND 100),
City VARCHAR(40)
);
```

Basic SQL commands

- **INSERT:** INSERT INTO Student (StudentID, Name, Marks) VALUES (1, 'Asha', 85);
- **Read:** SELECT Name, Marks FROM Student;
- **Filter:** SELECT * FROM Student WHERE Marks >= 70;
- **Sort:** SELECT * FROM Student ORDER BY Marks DESC;
- **Update:** UPDATE Student SET Marks = 90 WHERE StudentID = 1;
- **Delete rows:** DELETE FROM Student WHERE StudentID = 1;
- **Add a column:** ALTER TABLE Student ADD Email VARCHAR(100);
- **Remove a table:** DROP TABLE Student;

WHERE operators

- Comparison: =, <>, <, >, <=, >=.
- Logical: AND, OR, NOT.
- Range: BETWEEN low AND high.
- Set membership: IN (...).
- Pattern matching: LIKE; % matches zero or more characters and _ matches one character in standard SQL patterns.
- NULL checking: IS NULL or IS NOT NULL. Do not use = NULL.

Aggregate functions

- COUNT(): counts rows or non-NULL values depending on the expression.
- SUM(): total of numeric values.
- AVG(): average of non-NULL numeric values.
- MIN() / MAX(): minimum / maximum value.
- GROUP BY groups rows; HAVING filters groups after aggregation.

Chapter 8: SQL – Joins, Subqueries, Views and Indexes

Joins

- INNER JOIN: returns matching rows from both tables.
- LEFT JOIN: all rows from left table plus matching right rows; unmatched right columns are NULL.
- RIGHT JOIN: all rows from right table plus matches from the left.
- FULL OUTER JOIN: includes matched and unmatched rows from both sides, where supported.
- CROSS JOIN: Cartesian product.
- SELF JOIN: joins a table to itself using aliases.

Example: `SELECT s.Name, d.DepartmentName FROM Student s JOIN Department d ON s.DepartmentID = d.DepartmentID;`

Subqueries

- A subquery is a query nested inside another SQL statement.
- IN checks membership in a result set.
- EXISTS checks whether a subquery returns any row.
- A correlated subquery refers to values from the outer query.
- ANY and ALL compare a value against a set of values.

Views

- A view is a named query presented like a virtual table.
- Views can simplify queries and restrict which columns/rows users see.
- A normal view usually stores the query definition, not a separate copy of the result; a materialized view stores results where supported.

Indexes

- An index can speed up searches, joins and sorting.
- Indexes use extra storage and can slow INSERT, UPDATE and DELETE because they must be maintained.
- Common index structures include B-trees/B+ trees and hash indexes, depending on the DBMS.

Other SQL concepts

- Aliases: AS gives a temporary name to a column or table.
- DISTINCT removes duplicate result rows.
- UNION combines compatible results and removes duplicates; UNION ALL keeps duplicates.
- CASE expresses conditional values in a query.
- LIMIT / FETCH / TOP restrict result counts; syntax depends on the DBMS.

Chapter 9: Functional Dependencies and Normalization

Normalization organizes relations to reduce redundancy and prevent insertion, update and deletion anomalies.

Functional dependency (FD)

$X \rightarrow Y$ means that each value of attribute set X determines exactly one value of attribute set Y in the relation.

Key concepts

- Full functional dependency: Y depends on the whole composite determinant, not just part of it.
- Partial dependency: a non-prime attribute depends on a proper subset of a candidate key.
- Transitive dependency: $X \rightarrow Y$ and $Y \rightarrow Z$ imply $X \rightarrow Z$, with relevant non-key dependency conditions.
- Prime attribute: belongs to at least one candidate key.

Normal forms

- **1NF:** each attribute contains atomic values under the chosen relational model; no repeating groups.
- **2NF:** in 1NF and no non-prime attribute is partially dependent on any candidate key.
- **3NF:** for every nontrivial FD $X \rightarrow A$, X is a superkey or A is a prime attribute.
- **BCNF:** for every nontrivial FD $X \rightarrow Y$, X is a superkey.
- **4NF:** eliminates nontrivial multivalued dependencies unless the determinant is a superkey.
- **5NF:** addresses nontrivial join dependencies that are not implied by candidate keys.

Anomalies

- Insertion anomaly: cannot add one fact without another unrelated fact.
- Update anomaly: the same fact must be changed in multiple rows.
- Deletion anomaly: deleting one row accidentally removes another fact.

Decomposition goals

- Lossless join: joining decomposed relations reconstructs exactly the original relation.
- Dependency preservation: dependencies can be enforced without joining decomposed relations.
- 3NF synthesis can preserve dependencies; BCNF decomposition may not preserve every dependency.

Chapter 10: Transactions and ACID Properties

A transaction is a logical unit of work that may contain one or more database operations.

ACID properties

- **Atomicity:** all operations occur or none do.
- **Consistency:** a transaction takes the database from one valid state to another, preserving constraints.
- **Isolation:** concurrent transactions are controlled so their effects meet the selected isolation guarantees.
- **Durability:** committed changes survive failures according to the system's guarantees.

Transaction states

- Active: transaction is executing.
- Partially committed: final statement has executed but commit is not yet fully durable.
- Committed: transaction completed successfully.
- Failed: transaction cannot proceed normally.
- Aborted: effects are rolled back; the transaction may be restarted or ended.

Transaction control commands

- BEGIN / START TRANSACTION: begin a transaction (syntax varies).
- COMMIT: make transaction changes permanent.
- ROLLBACK: undo uncommitted changes.
- SAVEPOINT: create a point for partial rollback where supported.

Chapter 11: Concurrency Control and Schedules

Why concurrency control is needed

- Prevents lost updates, dirty reads, non-repeatable reads and inconsistent results.
- Serial schedule: transactions execute one after another.
- Concurrent schedule: operations from transactions are interleaved.
- Serializability: a concurrent schedule is equivalent to some serial schedule.

Conflicts and serializability

- Two operations conflict if they belong to different transactions, access the same data item, and at least one is a write.
- Conflict serializability can be tested using a precedence graph: a cycle means the schedule is not conflict-serializable.
- View serializability is a broader concept than conflict serializability.

Locking

- Shared (S) lock: permits reading; multiple shared locks can coexist.
- Exclusive (X) lock: permits writing and excludes incompatible locks.
- Two-phase locking (2PL): growing phase acquires locks; shrinking phase releases locks.
- Strict 2PL holds exclusive locks until commit/abort; this helps avoid cascading rollbacks.
- Deadlocks can occur in locking systems and may be detected or prevented.

Timestamp ordering and optimistic control

- Timestamp ordering uses transaction timestamps to order conflicting operations.
- Optimistic concurrency control generally uses read, validation and write phases; it assumes conflicts are relatively uncommon.

Isolation levels (common SQL terminology)

- Read Uncommitted: may allow dirty reads.
- Read Committed: prevents dirty reads; repeated reads may differ.
- Repeatable Read: prevents non-repeatable reads under the standard model; phantom behavior depends on implementation/standard details.
- Serializable: strongest standard isolation level; aims for results equivalent to serial execution.

Chapter 12: Database Recovery

Failure types

- Transaction failure: logical error, constraint violation or explicit abort.
- System crash: volatile state is lost, while persistent storage may remain.
- Media failure: storage damage or loss.
- Communication failure: relevant in distributed database systems.

Log-based recovery

- A log records transaction events such as start, update, commit and abort.
- Write-ahead logging (WAL): log records must reach stable storage before the corresponding changed data page is written, according to the recovery protocol.
- UNDO reverses effects of uncommitted transactions; REDO reapplies committed effects when required.
- Checkpointing records a recovery point and can reduce recovery work.

Other recovery techniques

- Deferred update: database changes may be applied after commit; redo may be required.
- Immediate update: changes can reach storage before commit; undo and/or redo may be required.
- Shadow paging maintains alternate page mappings instead of relying solely on an update log.
- Backups and restore procedures are important for media failures.

Chapter 13: Indexing, Hashing and Query Processing

Indexing

- Primary index: associated with the ordering/key arrangement of a data file in traditional textbook classifications.
- Clustering index: organizes records around an ordering field that may not be unique.
- Secondary index: provides an additional access path.
- Dense index: has an index entry for every search-key value or record, depending on the design.
- Sparse index: has entries for only some search-key values and usually relies on ordered data.

B-tree and B+ tree

- Balanced search trees keep search, insertion and deletion efficient.
- B+ tree internal nodes guide search; leaf nodes contain data entries or record pointers and are often linked for range scans.
- B+ trees are widely used in database indexes.

Hashing

- Hash function maps a search key to a bucket.
- Static hashing uses a fixed bucket structure.
- Dynamic hashing can grow or shrink its structure as data changes.
- Hash indexes are good for equality lookups but generally do not support ordered range scans as effectively as tree indexes.

Query processing

- Parsing and validation check SQL syntax and names.
- Translation converts the query into an internal representation.
- Optimization chooses an efficient execution plan.
- Execution runs the chosen plan.
- Optimization considers join order, available indexes, selectivity, estimated row counts and I/O/CPU costs.

Chapter 14: Distributed Databases and NoSQL

Distributed database

- A distributed database stores logically related data across multiple sites or nodes.
- Advantages may include availability, locality and scalability; challenges include coordination, consistency and network failures.
- Replication stores copies at multiple nodes; partitioning/sharding divides data among nodes.
- Distributed transactions may use protocols such as two-phase commit (2PC), which coordinates commit but can block under some failures.

CAP theorem (simplified)

During a network partition, a distributed data system must make a trade-off between strong consistency and availability for the affected operations. CAP is specifically about behavior when partitions occur, not a claim that every system simply chooses any two properties under all conditions.

NoSQL database types

- Key-value: stores values by keys; example category: Redis.
- Document: stores document-like records; example category: MongoDB.
- Wide-column: stores data in column families; example category: Cassandra.
- Graph: models nodes and edges; example category: Neo4j.

SQL vs NoSQL

- Relational databases commonly use tables, relations, constraints and SQL.
- NoSQL is a broad family of data models, often chosen for particular scalability, flexible-schema or graph/document access needs.
- Neither approach is universally better; choose based on data relationships, consistency requirements, queries and operational needs.

Chapter 15: Database Security and Administration

Security controls

- Authentication verifies identity; authorization checks permissions.
- Principle of least privilege limits each user to required access.
- GRANT and REVOKE manage privileges in SQL systems.
- Views can restrict visible data; row/column-level controls depend on the DBMS.
- Parameterized queries help prevent SQL injection; avoid building SQL by concatenating untrusted input.
- Encryption in transit and at rest protects data confidentiality.
- Audit logs support monitoring and investigations.

DBA responsibilities

- Install, configure and upgrade database software.
- Create users, roles and permissions.
- Plan backups and test restoration.
- Monitor performance, queries, indexes and storage.
- Manage availability, replication, recovery and capacity.
- Apply patches and follow security practices.

Chapter 16: Practical SQL Revision

Grouping and filtering

```
SELECT Department, AVG(Marks) AS AverageMarks
FROM Student
WHERE Marks IS NOT NULL
GROUP BY Department
HAVING AVG(Marks) >= 60
ORDER BY AverageMarks DESC;
```

Find duplicates

```
SELECT Name, COUNT(*)
FROM Student
GROUP BY Name
HAVING COUNT(*) > 1;
```

Subquery example

```
SELECT Name FROM Student
WHERE Marks > (SELECT AVG(Marks) FROM Student);
```

Create a view

```
CREATE VIEW HighScorers AS
SELECT StudentID, Name, Marks FROM Student
WHERE Marks >= 80;
```

Important SQL reminders

- Use WHERE to filter rows before grouping and HAVING to filter groups.
- Use ORDER BY for sorting; SQL result order is not guaranteed without it.
- Use explicit JOIN ... ON syntax for readable joins.
- UPDATE and DELETE should normally include a carefully checked WHERE clause.
- Exact features and syntax differ among MySQL, PostgreSQL, Oracle, SQL Server and SQLite.

Chapter 17: Important Exam Questions

Short-answer questions

- Define DBMS and list its advantages.
- Differentiate file processing system and DBMS.
- Explain three-schema architecture and data independence.
- Define schema, instance, entity, attribute and relationship.
- Explain primary, candidate, super and foreign keys.
- Explain entity integrity and referential integrity.
- Describe relational algebra operations.
- Explain DDL, DML, DQL, DCL and TCL with examples.
- Write SQL queries using WHERE, GROUP BY, HAVING and ORDER BY.
- Explain different types of joins and subqueries.
- Define functional dependency and explain 1NF, 2NF, 3NF and BCNF.
- Explain ACID properties and transaction states.
- What are serializability and two-phase locking?
- Explain deadlock, log-based recovery, UNDO and REDO.
- Compare B-tree/B+ tree indexing and hashing.
- Explain distributed databases, replication, sharding and NoSQL types.
- Discuss database security and the duties of a DBA.

Practice checklist

- Draw an ER diagram and convert it into relational tables.
- Identify candidate keys and functional dependencies.
- Normalize a relation up to 3NF/BCNF.
- Write SELECT queries with joins, aggregates and subqueries.
- Trace transaction schedules and build a precedence graph.
- Practise lossless decomposition and dependency-preservation questions if they are in your syllabus.

Final study tip

For exam preparation, learn exact definitions, practise SQL by writing queries, draw ER diagrams, and solve normalization and transaction problems step by step. Use your university syllabus and past papers to confirm the required depth.